

FunBic-CCA: Function Secret Sharing for Biclusterings Applied to Cheng and Church Algorithm (Extended Version)^a

Shokofeh VahidianSadegh¹, Alberto Ibarrondo² and Lena Wiese¹

¹Computer Science Department, Goethe University Frankfurt, Frankfurt am Main, Germany

²Arcium, France

VahidianSadegh@mathematik.uni-frankfurt.de, ibarrond@eurecom.fr, lwiese@cs.uni-frankfurt.de

Keywords: Biclustering Algorithm, Privacy-Preserving AI, Secure Multi-Party Computation, Function Secret Sharing.

Abstract: High-throughput technologies (e.g., the microarray) have fostered the rapid growth of gene expression data collection. These biomedical datasets, increasingly distributed among research institutes and hospitals, fuel various machine learning applications such as anomaly detection, prediction or clustering. In particular, unsupervised classification techniques based on biclustering like the Cheng and Church Algorithm (CCA) have proven to adapt particularly well to gene expression data. However, biomedical data is highly sensitive, hence its sharing across multiple entities introduces privacy and security concerns, with an ever-present threat of accidental disclosure or leakage of private patient information. To address such threat, this work introduces a novel, highly efficient privacy-preserving protocol based on secure multiparty computation (MPC) between two servers to compute CCA. Our protocol performs operations relying on an additive secret sharing and function secret sharing, leading us to reformulate the steps of the CCA into MPC-friendly equivalents. Leveraging lightweight cryptographic primitives, our new technique named FunBic-CCA is first to exploit the efficiency of function secret sharing to achieve fast evaluation of the CCA biclustering algorithm.

1 INTRODUCTION

The abundance of biomedical data, thanks to the rapid advancement and readily available high-throughput technologies, provides great opportunities for the knowledge discovery. Machine Learning (ML) has been widely used to categorise and classify large amounts of data. Among ML methods, the traditional one-way clustering methods perform analysis directly on specific characteristics. More adaptable approaches to gene expression data are biclustering algorithms that have become prevalent with a wide variety of applications, ranging from bioinformatics to recommender systems, and many more. Biclustering based on *Cheng and Church Algorithm* (CCA) groups a set of genes and conditions with a high similarity score. This was the first study to apply biclustering algorithms over gene expression data (Cheng and Church, 2000).

Despite this potential of large biomedical data analysis, a recent influx of malicious attacks on these sensitive resources has been increasingly reported. Since that, there exists an inherent, simultaneous con-

tradiction between utility and privacy. Accordingly, there must be improved data protection and preventive information leakage measures against unauthorised users.

Considering this problem, we propose FunBic-CCA (**F**unction secret sharing for **B**iclusterings **A**ppplied to **C**heng and **C**hurch **A**lgorithm), a privacy-preserving framework for gene expression data analysis by biclustering algorithms with well-established schemes, *additive secret sharing* (SS (Demmler et al., 2015)) and *Function Secret Sharing* (FSS (Boyle et al., 2015)).

In our previous studies (VahidianSadegh et al., 2022), we presented feasibility of applying encryption schemes like BFV (Fan and Vercauteren, 2012) and CKKS (Cheon et al., 2017) over CCA, leaving out non-linear operations in the encrypted domain.

We develop FunBic-CCA in three main aspects:

- We construct a comparative platform for CCA as the foundation and proof-of-concept implementation of biclusterings.
- We put special care to balance accuracy and performance with security guarantees, while improving upon existing works in the domain.

^aThis is an extended version of the paper appeared in (VahidianSadegh et al., 2025)

- We consider security against a semi-honest adversary.

After discussing preliminaries from privacy-preserving biclustering analysis methods (Section 2), we detail FunBic-CCA framework (Section 3), which provides core contributions below:

- An end-to-end privacy-preserving framework based on additive secret sharing and function secret sharing schemes;
- Support for privacy-preserving evaluation of both linear and non-linear operations with 100% correctness in less than 500 seconds;
- An extensive discussion on the security aspects of FunBic-CCA as well as analyses on its accuracy and efficiency;
- An open-source implementation¹ of our framework, available under the permissive MIT license.

We evaluate the efficacy of FunBic-CCA through experiments and evaluation measures using our framework (Section 4). Finally, we present an overview of related works and elaborate on differences of existing solutions (Section 5) then conclude (Section 6).

2 BACKGROUND

We provide an overview of the applied techniques and methods in FunBic-CCA. We first define our notation and proceed with a description of Cheng and Church Algorithm (Cheng and Church, 2000). Afterwards, an overview of the cryptography methods used in our framework is discussed.

Notation. We use regular letters for scalars (e.g., s) and boldface letters for matrices in capitals ($\mathbf{A}$) and vectors of integers and polynomials (e.g., $\mathbf{a}$). a_{ij} denotes the element of the matrix $\mathbf{A}$ in the i -th row and j -th column of a polynomial/ vector $\mathbf{a}$ with N coefficients/ elements. We write $\mathbf{a} \cdot \mathbf{b} = \mathbf{c}$ to denote the element-wise two vector multiplication, while showing matrix multiplication like $\mathbf{A} \times \mathbf{B} = \mathbf{C}$.

In a 2PC paradigm, P_0, P_1 denote two computing parties or in general, we consider P_d , where $d \in \{0, 1\}$. a_{ij_d} is an element of expression matrix $\mathbf{A}$ for a party d . We use $P_a: q \Rightarrow P_b$ to show that party a sends value q to party b . We employ $\langle x \rangle$ to indicate that value x is arithmetically secret shared into random shares (x_0, x_1) , that are held by two computing parties and verify $x = x_0 + x_1 \mod$

2^n . All values are encoded on n -bits, which lives in $\mathbb{Z}_{2^n}$. $\mathbb{Z}_{n+}^*$ represents the range $0 \leq x \leq 2^{n-1} - 1$.

We note $\mathcal{U}_{2^n}^{[l_r, l_c]}$ as the uniform distribution in the set 2^n with the size of maximum number of elements in a matrix with l_r rows and l_c columns. We use $1_{r2del}, 1_{c2del}, 1_{r2sdel}, 1_{c2sdel}, 1_{r2add}, 1_{c2add}$ to elaborate on the comparison functions (e.g., $1_{x \geq 0} \Leftrightarrow x \geq 0$).

2.1 Biclustering Algorithms

Biclustering or co-clustering is an unsupervised learning task with which both the rows and columns can be placed in concurrent clusters to obtain coherent and homogeneous sub-matrices. Main advantage of biclustering remains in detecting physiologically relevant and computationally significant genes that better reflect gene relationships underlying cellular processes or molecular function compared to other classification techniques (Ali et al., 2023).

Biclustering was initially introduced by Hartigan in the 1970s (Hartigan, 1972), whilst Cheng et al. were first to apply biclustering algorithms over gene expression data to discover co-expressed genes (Cheng and Church, 2000).

Cheng and Church Algorithm (CCA). Cheng and Church (Cheng and Church, 2000) developed a greedy algorithm and used a measure to evaluate the consistency of matrix's elements and to assess the quality of a bicluster. Mean square residue score (MSR) measures the coherence of genes and conditions of a bicluster by taking means of genes and conditions. CCA focuses mainly on particularly large, maximal biclusters with score below a predefined threshold (δ). In other words, given that a matrix $\mathbf{A}$ and a threshold $\delta \geq 0$, CCA finds δ -biclusters. After finding values of the mean for rows (μ_r), columns (μ_c) and all of elements (μ_{ij}), the residue of the entry a_{ij} for a bicluster $\mathbf{B}_k = (r_k, c_k)$ is

$$r_{ij} = a_{ij} - \mu_r - \mu_c + \mu_{ij}. \quad (1)$$

which is needed to determine MSR (or H_{ij}):

$$H_{ij} = \frac{\sum_{i \in r_k} \sum_{j \in c_k} r_{ij}^2}{|r_k| |c_k|}. \quad (2)$$

Score for rows, and columns separately can be summarised as follow in which:

$$H_r = \frac{1}{|c_k|} \sum_{j \in c_k} r_{ij}^2, H_c = \frac{1}{|r_k|} \sum_{i \in r_k} r_{ij}^2. \quad (3)$$

The three different phases in CCA (see algorithm 1) include multiple node deletion, single node deletion and node addition. First step is the removal

¹<https://github.com/ShokofehVS/FunBic-CCA>

of multiple rows/columns over the input data matrix, which is followed by the single row/column deletion step. At this point, the result may not be maximal and, therefore, a node addition step is required to insert some rows/columns without increasing MSR. CCA takes as input, the expression matrix ($\mathbf{A}$), maximum acceptable mean squared residue score threshold (δ), a parameter for multiple node deletion step (α), and number of δ -biclusters (k) (Cheng and Church, 2000).

2.2 Secure Multi-Party Computation (MPC)

Secure Multi-Party Computation (MPC) is a class of cryptographic protocols which allows two or more parties to compute any function over shared private input data without revealing any information except for output of the function (Ibarrondo et al., 2023). Formally, let $P_1, P_2, \dots, P_g$ be g parties; each has an input $q_1, q_2, \dots, q_g$ and wants to jointly compute the function f , $f(q_1, q_2, \dots, q_g) = (o_1, o_2, \dots, o_g)$. Given that party g is guaranteed to learn o_g , but can get nothing more than that (Catalano et al., 2005).

The idea of secure computation backs to the early 1980s introduced by Andrew Yao (Yao, 1982). Afterwards, the problem in secure two-party computation (2PC) was generalised to multiple parties in (Goldreich et al., 1987). In the field, a common approach relies on a model called preprocessing sometimes referred to as the offline-online paradigm including two phases.

Further, we describe our used MPC techniques, Additive Secret Sharing (SS), and Function Secret Sharing (FSS) with a focus on 2PC scenario because of its simplicity and efficiency.

Additive Secret Sharing. Additive secret sharing (SS) focuses on rings with distributed secret x into two random shares x_0 and x_1 , given that $x = x_0 + x_1 \bmod N$ (where N is the ring size) among two computing parties P_d . Since then, two parties are able to perform local addition/subtraction of two secret shared values. However, parties require one round of communication to do multiplication by having Beaver's multiplication triples (Beaver, 1992). Lastly, the result is reconstructed by one chosen party, which adds the two secret shares together. We work with $N = 2^n$, where $n \in \{8, 16, 32, 64\}$, by considering native integer types in modern computers that increase the speed in working with the n -bit modular arithmetic.

Function Secret Sharing. A 2PC Function Secret Sharing (FSS) scheme unlike standard secret sharing

Algorithm 1: CCA($\mathbf{A}, \delta, \alpha, k$) $\rightarrow k$ δ -biclusters

```

1 Input:  $\mathbf{A}$ : matrix of real numbers ( $\in \mathbb{R}^{l_r \times l_c}$ ).
2    $\delta$ : maximum acceptable MSR score ( $\geq 0$ ).
3    $\alpha$ : multiple node deletion threshold ( $\geq 1$ ).
4    $k$ : number of  $\delta$ -biclusters to be found.
5
6 Output:  $k$   $\delta$ -biclusters in  $\mathbf{A}$ 
7
8 Initialisation: Replace missing elements in  $\mathbf{A}$  with random numbers in a uniform distribution.  $\mathbf{A}'$  is a copy of  $\mathbf{A}$ .
9
10 Iteration: for  $k$  times:
11   Multiple node deletion step( $\mathbf{A}', \alpha, \delta$ )  $\rightarrow \mathbf{B}$ :
12      $\mu_c, \mu_r, \mu_{ij}$  and  $H_{ij}$ ;
13     if  $H_{ij} \leq \delta$  then
14       | return  $\mathbf{A}'$ ;
15     while  $H_{ij} > \delta$  do
16       |  $\mathbf{B} = \mathbf{A}'$ ;
17       | if  $H_r > \alpha H_{ij}$  then
18         |  $A'_i = 0$ ;
19         |  $\mu_c, \mu_{ij}$  and  $H_{ij}$ ;
20       | else
21         | if  $H_c > \alpha H_{ij}$  then
22           |  $A'_j = 0$ ;
23         | else
24           | return  $\mathbf{B}$ ;
25     return  $\mathbf{B} = \mathbf{A}'$ ;
26   Single node deletion step( $\mathbf{B}', \delta$ )  $\rightarrow \mathbf{C}$ :
27     while  $H_{ij} > \delta$  do
28       |  $\mu_c, \mu_r, \mu_{ij}$  and  $H_{ij}$ ;
29       |  $d(i) = \max_i(H_r)$ ;
30       |  $d(j) = \max_j(H_c)$ ;
31       | if  $H_r(d(i)) \geq H_c(d(j))$  then
32         |  $B'_i = 0$ ;
33       | else
34         |  $B'_j = 0$ ;
35     return  $\mathbf{C} = \mathbf{B}'$ ;
36   Node addition step( $\mathbf{C}'$ )  $\rightarrow \mathbf{D}$ :
37     while ( $D_i \neq C'_i \wedge D_j \neq C'_j$ ) do
38       |  $\mathbf{D} = \mathbf{C}'$ ;
39       |  $\mu_c, \mu_r, \mu_{ij}$  and  $H_{ij}$ ;
40       | if  $H_c \leq H_{ij}$  then
41         |  $C'_j = 1$ ;
42       | if ( $H_r \leq H_{ij} \vee H_r^{-1} \leq H_{ij}$ ) then
43         |  $C'_i = 1$ ;
44     return  $\mathbf{D} = \mathbf{C}'$ ;

```

of individual elements, shares description of a function f among parties (Boyle et al., 2015). In this scheme, f is split into two shares (f_0, f_1) , where f_d

hides f and $f_0(x) + f_1(x) = f(x)$ for the input value x that is public for both parties. Due to the publicity of the secret input x , a random mask r is added $\hat{x} = x + r$ before using in FSS evaluation to keep its privacy. The used mask is generally known by dealer and applied for the key generation to blind the input. According to MPC preprocessing model, a FSS gate encompasses two algorithms:

- $\text{Gen}(1^\lambda, f) \rightarrow (\mathbf{k}_0, \mathbf{k}_1)$: is a key generation algorithm that outputs a pair of function keys $(\mathbf{k}_0, \mathbf{k}_1)$ with descriptions for f_0, f_1 , input mask shares r_0, r_1 given the security parameter λ , and the function's description $f : \mathbb{G}_{in} \mapsto \mathbb{G}_{out}$.
- $\text{Eval}(d, \mathbf{k}_d, \hat{x}) \rightarrow f(x)$: is a polynomial-time deterministic algorithm that outputs an additive share $f_d(\hat{x})$ where $f_0(\hat{x}) + f_1(\hat{x}) = f(x)$ given party index d , the function key $\mathbf{k}_d$ and the masked input $\hat{x}$.

We use Interval Containment (IC) gate for comparison ($\geq \gamma, \gamma = 0$) in our work. IC gate in FSS scheme computes $f_{p,q}(x) = 1_{x \in [p,q]}$ where $p = 0, q = 2^{n-1} - 1$ to finally obtain $1_{0 \leq x \leq 2^{n-1} - 1}$ (Section 4.1 of (Boyle et al., 2021)). A central building block of FSS is the Distributed Comparison Function (DCF) (Gilboa and Ishai, 2014), that determines a family of comparison functions $f_{\alpha,\beta}^<$. It outputs β if $x > \alpha$ and 0 otherwise. FSS gate for IC is built on top of two evaluations of DCF block. Later, Boyle et al. introduced a new method to construct IC using a single DCF block (Fig. 3 of Section 4.1 (Boyle et al., 2021)). Our equality check between two secret shares is relied on the Distributed Point Function (DPF) (Definition 2.5 of (Boyle et al., 2019)) as an FSS scheme for the family of all point functions. This states that for a point function $f_{\alpha,\beta}, f(\alpha) = \beta$ and $f(x) = 0$ for $x \neq \alpha$, where $\alpha \in \mathbb{G}^{in}, \beta \in \mathbb{G}^{out}, f : \mathbb{G}^{in} \rightarrow \mathbb{G}^{out}$.

3 FUNBIC-CCA FRAMEWORK

In this Section, we provide a detailed description of our FunBic-CCA framework.

Sketch of the Solution. We consider a scenario by having a data owner and two cloud servers as our computing parties (P_0, P_1):

- *Data owner*: or patient with his genes sequenced as matrix $\mathbf{A}$ of expression level values a_{gp} , consisting of genes $\mathbf{g}$ under different conditions $\mathbf{p}$.
- *Public cloud servers*: by which we intend to execute biclustering on the secret shared gene expression data with our algorithmic design.

Table 1: Steps within Cheng and Church Algorithm.

Step	Description
1	Finding mean squared residue scores H_r, H_c and H_{ij}
2	Evaluation in multiple node deletion $H_r > \alpha H_{ij}, H_c > \alpha H_{ij}$
3	Evaluation in single node deletion $H_r[d(i)] \geq H_c[d(j)], H_r[d(i)] < H_c[d(j)]$
4	Evaluation in node addition $H_r \leq H_{ij}, H_c \leq H_{ij}$

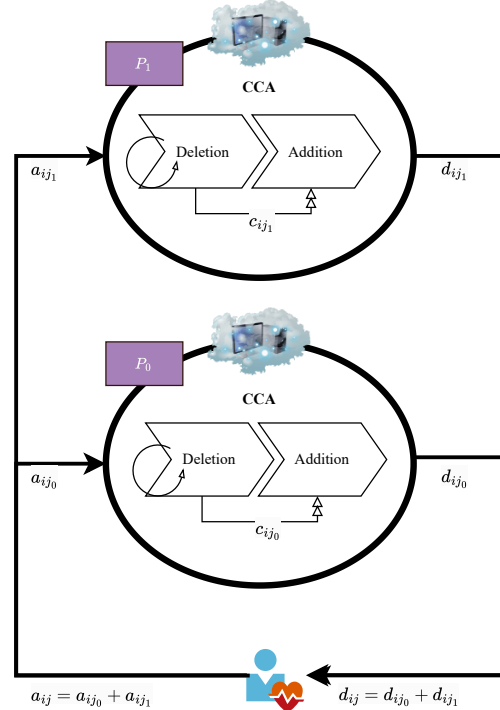


Figure 1: General overview of FunBic-CCA system.

Different types of computations are needed in the procedure that can potentially be transferred to the public cloud services. Table 1 contains these steps (further explanations on these steps in Section 2.1) and Figure 1 represents the general overview of our proposed solution. Prior to the introduction of our main building blocks, we summarise the main generic functions inside our algorithm (with the steps in Table 1) that we intend to protect by MPC based operations (see Section 3.1):

$$\begin{aligned}
& H_{ij}, H_r, H_c \\
& H_r > \alpha H_{ij}, H_c > \alpha H_{ij} \\
& H_r[d(i)] \geq H_c[d(j)], H_r[d(i)] < H_c[d(j)] \\
& H_r \leq H_{ij}, H_c \leq H_{ij}
\end{aligned} \tag{4}$$

Two-Party Computation Scenario. Our solution is based on the two-party computation (2PC) context, having two computing parties (P_0, P_1) to perform secure computations as well as the data owner. In this two-server protocol, the data owner interacts with these two servers that are assumed not to collude (Boyle et al., 2021).

Our framework is secure against Honest-but-Curious (“semi-honest”) adversaries. Therefore, FunBic-CCA provides privacy of all the input data in the case of corruption of either the two parties by a semi-honest adversary. This work is designed in MPC with a preprocessing model, having a setup/offline phase to benefit from an optimised online phase.

3.1 Cryptographic Primitives

Here, we provide details of our building blocks that form the core of our framework and for which we decide to apply secret sharing schemes. In all operations, d refers to the number of parties, precisely $d \in \{0, 1\}$. Note that, each individual element (a_{ij}) of the input matrix ($\mathbf{A}$) is secret shared among parties with the following condition: $a_{ij} = a_{ij_0} + a_{ij_1}$.

Construction of Secure Linear Operations. We require to obtain the below-mentioned operations, such as addition/subtraction, and multiplication:

- Addition/ subtraction: These operations can be computed locally. Accordingly, both parties are able to perform the following operations on their own secret shared input matrix.
- Finding the mean values, residue, scores of the whole matrix, and nodes are based on addition/ subtraction.

$$\begin{aligned}
& \sum_{l_r \cdot l_c} a_{ij_d}, \sum_{l_r} a_{ij_d}, \sum_{l_c} a_{ij_d} \\
& \sum_{l_r \cdot l_c} (a_{ij_d} - \mu_{r_d} - \mu_{c_d} + \mu_{ij_d}) \\
& \sum_{l_r \cdot l_c} r_{ij_d}^2, \sum_{l_r} r_{ij_d}^2, \sum_{l_c} r_{ij_d}^2
\end{aligned} \tag{5}$$

- Multiplication: Both parties perform SS-based multiplications in one round of communication

and consumption of the beaver multiplication triples.

- Squaring the residue ($r_{ij_0} \times r_{ij_1}$) as part of the score formula requires parties to perform element-wise multiplication for the matrices jointly using beaver triples. We follow the steps mentioned in (Knott et al., 2021). To square the residue $r_{ij}^2 = r_{ij_0}^2 + r_{ij_1}^2 + 2 \times r_{ij_0} \times r_{ij_1}$, the parties use a beaver pair $\hat{a}, \hat{b}$, such that $\hat{b} = \hat{a}^2$. Then, parties compute $\hat{e}_d = r_{ij_d} - \hat{a}_d$, decrypt $\hat{e}_d$ to obtain result with $r_{ij}^2 = \hat{b}_d + e^2 + 2 \times e \times \hat{a}_d$.

Construction of Secure Non-linear Operations. Our algorithm relies on non-linear operations, including comparison, argmax and division:

- Comparison: Main comparative functions are denoted in Table 2. The threshold in comparison i.e., zero (is referred to γ), must be kept private and hidden by parties by using a random mask. In the online phase, the parties are required to compute

$$\begin{aligned}
& 1_{r2del(H_{r_d}, \alpha \cdot H_{ij_d}) > 0}, 1_{c2del(H_{c_d}, \alpha \cdot H_{ij_d}) > 0} \\
& 1_{r2sdel(H_{r_d}[d(i)], H_{c_d}[d(j)]) \geq 0}, 1_{c2sdel(H_{c_d}[d(j)], H_{r_d}[d(i)]) > 0} \\
& 1_{r2add(H_{ij_d}, H_{r_d}) \geq 0}, 1_{c2add(H_{ij_d}, H_{c_d}) \geq 0} \\
& 1_{stop(H_{ij_d}, \delta) \geq 0} \\
& 1_{req(I_d, I'_d) = 0}, 1_{ceq(J_d, J'_d) = 0}
\end{aligned} \tag{6}$$

on the secret shares a_{ij_d} .

Parties evaluate IC gate to determine whether the result is greater or equal to the threshold for $1_{r2del}, 1_{c2del}, 1_{r2sdel}, 1_{c2sdel}, 1_{r2add}, 1_{c2add}$. The DPF gate handles our equality checks, $1_{req}, 1_{ceq}$.

For stopping condition 1_{stop} , we reconstruct the intermediate results o_{stop_0}, o_{stop_1} after FSS IC gate to check whether our score H_{ij_d} , is below or equal to the threshold δ .

To this end, our condition to remove columns, 1_{con} , is done in the cleartext, since the size of the input is public and known by both parties. Note that, each of the comparison function requires one round of communication and to send two ring elements in the online phase.

- Argmax: Single node deletion step works with removing a single row or column, whichever having the largest $d(i)$ or $d(j)$. This is achieved by finding the argmax of rows and columns with the largest score:

$$d(i) = \text{argmax}(H_r), d(j) = \text{argmax}(H_c) \tag{7}$$

Our argmax function is similar to the algorithm 6 of (Ryffel et al., 2020), using pairwise comparisons. It gets as input; secret shared scores of rows

Table 2: FunBic-CCA comparative functions.

Formula	Symbol
$H_r > \alpha H_{ij}, H_c > \alpha H_{ij}$	$1_{r2del}, 1_{c2del}$
$H_r[d(i)] \geq H_c[d(j)]$	$1_{r2sdel}, 1_{c2sdel}$
$H_r \leq H_{ij}, H_c \leq H_{ij}$	$1_{r2add}, 1_{c2add}$
$H_{ij} \leq \delta$	1_{stop}
$I == I' \wedge J == J'$	$1_{req}, 1_{ceq}$
$ J \leq 100$	1_{con}

or columns (H_r, H_c) and outputs the index of the row or column with the largest score value.

Algorithm 4 requires a constant number of rounds. For instance, argmax of rows with length l_r requires $l_r(l_r - 1)$ parallel comparisons with IC gate and l_r equality checks with DPF gate. Therefore, we need two rounds of communication and sending $O(l_r^2)$ values in an online phase.

- Division: Finding mean which is a base function for calculation of the scores relies on a division.

$$\frac{\sum^{l_r \cdot l_c} a_{ij_d}}{|I||J|}, \frac{\sum^{l_r} a_{ij_d}}{|J|}, \frac{\sum^{l_c} a_{ij_d}}{|I|}$$

$$\frac{\sum^{l_r \cdot l_c} r_{ij_d}^2}{|I||J|}, \frac{\sum^{l_r} r_{ij_d}^2}{|J|}, \frac{\sum^{l_c} r_{ij_d}^2}{|I|} \quad (8)$$

In secured machine learning applications (Ryffel et al., 2020), it is a standard assumption to keep the data and model parameters private; however, the shape (number of rows and columns) of the input and the architecture of the model are public. In addition, multiplying or dividing by a public value (Escudero, 2022) such as $|I|, |J|$ can be executed locally.

Prior to any changes on nodes (removing, adding), the dimension of matrix is clear; thus the mean values are being done as mentioned above. Additionally, parties do local operations, including addition/ subtraction simultaneously by one, once any node is added/ removed.

3.2 System Protocol

We describe our protocols needed to deliver the full solution based on the cryptographic primitives in Section 3.1. Figure 2 shows these protocol steps.

We follow the below settings in the offline and on-line phases, having the two parties (P_0, P_1):

- Offline phase: Correlated randomness is required to perform the secured multiplication according to the discussion in Section 3.1. The generation of the FSS keys for the comparison functions also

occurs in “FunBic-CCA.setup”. These preprocessing materials are distributed among the parties to be used in the online phase.

We assume a trusted dealer, an individual entity, to only participate in the offline phase and provide the computing parties with the preprocessing materials. We can also realise a trusted dealer by ❶ placing a Trusted Execution Environment (TEE, such as Intel SGX) inside one of the computing parties (Nawaz et al., 2020), ❷ designing a semi-honest 3PC in an honest majority setting (third party will be inactive during online phase) (Wagh, 2022), or ❸ having our two computing parties in a pure 2PC scenario jointly generate required preprocessing materials (Boyle et al., 2021). To this end, we resort to either Oblivious Transfer (OT) or Homomorphic Encryption (HE) (Demmler et al., 2015) for the SS preprocessing materials and generic 2PC technique for the FSS keys (Appendix A.2 of (Boyle et al., 2021)) by the trusted dealer.

Algorithm 2:

FunBic-CCA.setup ($l_r, l_c, n, \lambda, \gamma$) $\rightarrow K_0, K_1$.

Input: l_r, l_c : length of matrix **A**.

n : number of bits for secret sharing ring $\mathbb{Z}_{2^n}$.

λ : security parameter.

γ : threshold for comparison ($\in \mathbb{Z}_{2^n}, \gamma = 0$).

Output: K_0, K_1 : keys in preprocessing phase.

Preprocessing Steps:

Beaver triples for multiplication:

- ❶ $\langle \hat{a} \rangle \equiv (\hat{a}_0, \hat{a}_1) \sim \mathcal{U}_{2^n}^{l_r \cdot l_c}$
- ❷ $\hat{b}_0 \leftarrow \hat{a}_0^2 \sim \mathcal{U}_{2^n}^{l_r \cdot l_c}$
- ❸ $\hat{b}_1 \leftarrow (\hat{a}_0 + \hat{a}_1)^2 - \hat{b}_0$
- ❹ $\langle \hat{b} \rangle \equiv (\hat{b}_0, \hat{b}_1)$

Random masks for FSS gates:

- ❶ $\langle r \rangle \equiv (r_0, r_1) \leftarrow (r_0 + r_1) \sim \mathcal{U}_{2^n}^2$
- ❷ $\langle r_\gamma \rangle \equiv (r_{\gamma_0}, r_{\gamma_1}) \leftarrow (r_0, r_1 - \gamma)$

Distribution of preprocessing materials:

- ❶ $K_d \equiv (\hat{a}_d, \hat{b}_d, r_{\gamma_d}, K_d^{IC}, K_d^{EQ}), d \in \{0, 1\}$
- ❷ $K_0 \Rightarrow P_0, K_1 \Rightarrow P_1$

- Online phase:

1. Identification of the scores: Calculation of the mean squared residue for the whole input matrix and its rows and columns (i.e., $H_{ij_d}, H_{r_d}, H_{c_d}$) takes place in “FunBic-CCA.MSR”.

The parties perform linear operations, addition/ subtraction, along with a non-linear division locally. Further, the parties square the residue

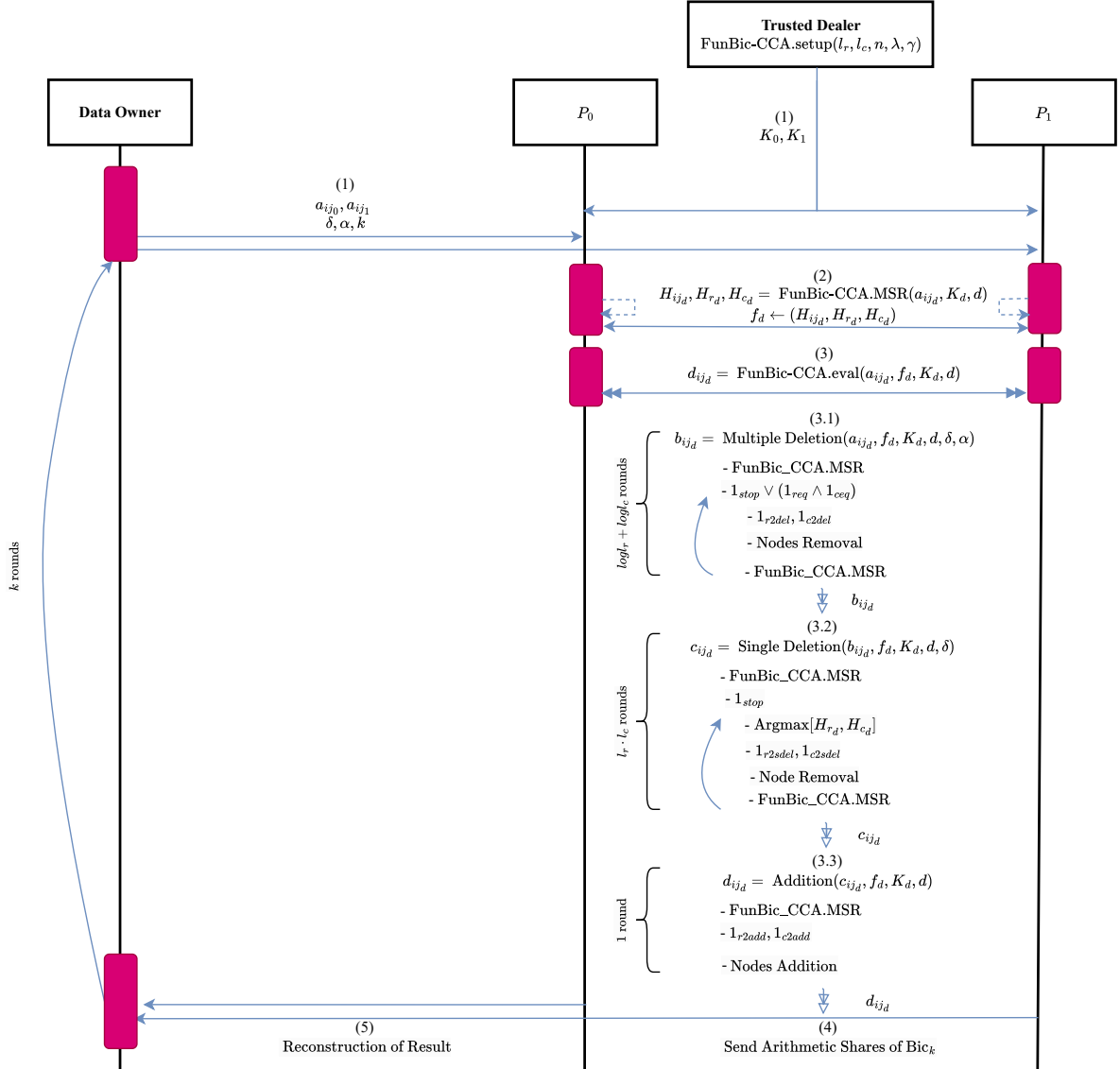


Figure 2: System diagram of the end-to-end secured computation of CCA using FunBic-CCA's algorithm.

(r_{ij_d}) by having the correlated randomness from the setup phase in one round of communication.

2. Deletion/ addition of the nodes: Parties do the comparisons, composing of $1_{r2del}, 1_{c2del}$ inside the multiple node deletion, $1_{r2sdel}, 1_{c2sdel}$ of the single node deletion, $1_{r2add}, 1_{c2add}$ in the node addition, and the stopping function 1_{stop} as explained in Table 2, by performing the "FunBic-CCA.eval" with the IC gate (i.e., we keep $\text{FSS.Gen}^{IC}, \text{FSS.Eval}^{IC}$ calls to the original protocols 1, and 2 in (Ibarrondo et al., 2023)).

Further, the parties benefit from the DPF gate (i.e., construction of $\text{FSS.Gen}^{EQ}, \text{FSS.Eval}^{EQ}$

for equality check maintains a call to DPF gate (Boyle et al., 2019)) to obtain $1_{req}, 1_{ceq}$ and to also decide which nodes have to be removed/ added in the deletion and addition steps. Deletion of a single node depends on the result of the argmax, which invokes both FSS gates for the comparison and equality check. To this end, parties can locally increase/ decrease with one simultaneously to track the actual size of the secret shared matrix (i.e., some rows/ columns are masked with zeros in the deletion steps). The resulting matrices are sent back to the next step.

3. Final result: Parties return their final output to the data owner for its reconstruction in the

Algorithm 3:

FunBic-CCA.MSR $(a_{ij_d}, K_d, d) \rightarrow H_{ij_d}, H_{r_d}, H_{c_d}$.

Input: a_{ij_d} : input matrix

$(\in \{a_{ij_0}, a_{ij_1}\}, \mathbb{Z}_{2^n}^{l_r \cdot l_c})$.

K_d : preprocessing keys containing $\hat{a}_d, \hat{b}_d$.

d : computing parties $(\in \{0, 1\})$.

Output: $H_{ij_d}, H_{r_d}, H_{c_d}$: similarity scores $(\in l_r \cdot l_c)$.

Score Steps:

Local linear functions (addition and subtraction):

- ① $\sum^{l_r \cdot l_c} a_{ij_d}, \sum^{l_r} a_{ij_d}, \sum^{l_c} a_{ij_d}$
- ② $\sum^{l_r \cdot l_c} (a_{ij_d} - \mu_{r_d} - \mu_{c_d} + \mu_{ij_d})$
- ③ $\sum^{l_r \cdot l_c} r_{ij_d}^2, \sum^{l_r} r_{ij_d}^2, \sum^{l_c} r_{ij_d}^2$

Local non-linear function (division):

- ① $\frac{\sum^{l_r \cdot l_c} a_{ij_d}}{|I||J|}, \frac{\sum^{l_r} a_{ij_d}}{|J|}, \frac{\sum^{l_c} a_{ij_d}}{|I|}$
- ② $\frac{\sum^{l_r \cdot l_c} r_{ij_d}^2}{|I||J|}, \frac{\sum^{l_r} r_{ij_d}^2}{|J|}, \frac{\sum^{l_c} r_{ij_d}^2}{|I|}$

Reconstruction of $\hat{e}$ for multiplication:

- ① $\hat{e}_d \leftarrow (r_{ij_d} - \hat{a}_d); \langle \hat{e} \rangle \equiv (\hat{e}_0, \hat{e}_1) \sim \mathcal{U}_{2^n}^{l_r \cdot l_c}$
- ② $\hat{e}_d \Rightarrow P_{1-d}; \hat{e} \leftarrow \hat{e}_0 + \hat{e}_1$

Squared residue:

- ① $r_{ij_d}^2 = \hat{b}_d + e + 2 \times e \times \hat{a}_d$

Algorithm 4:

Argmax $(H_{r_d} \rightarrow \text{argmax}_{w \in [1, l_r]} H_{r_d})$.

Input: H_{r_d} : secret shared score for rows.

Output: $\text{argmax}_{w \in [1, l_r]} H_{r_d}$.

for $w \leftarrow 1$ **to** l_r **do**

```

1   for  $v \leftarrow 1$  to  $l_r$  do
2       if  $w \neq v$  then
3            $\hat{\gamma}_d \leftarrow H_{r_d}[w] - H_{r_d}[v] + r_{\gamma_d}$ 
4       Comparison of  $H_{r_d}[w] \geq H_{r_d}[v]$ :
5           ①  $\hat{\gamma}_d \Rightarrow P_{1-d}; \hat{\gamma} \leftarrow \hat{\gamma}_0 + \hat{\gamma}_1$ 
6           ②  $geq_d \leftarrow \text{FSS.Eval}^{IC}(d, K_d^{IC}, \hat{\gamma})$ 
7           ③  $s_d \leftarrow \sum^{l_r} geq_d$ 
8       Equality check on  $s_d == l_r - 1$ :
9           ①  $\hat{\gamma}_d \leftarrow s_d - (l_r - 1) + r_{\gamma_d}$ 
10          ②  $\hat{\gamma}_d \Rightarrow P_{1-d}; \hat{\gamma} \leftarrow \hat{\gamma}_0 + \hat{\gamma}_1$ 
11          ③  $eq_d \leftarrow \text{FSS.Eval}^{EQ}(d, K_d^{EQ}, \hat{\gamma})$ 
12      if  $eq_w == 1$  then
13          return  $w$ 

```

"FunBic-CCA.result".

3.3 Security Analysis

Overview. As explained earlier in Section 3, our framework is secure against Honest-but-Curious ("semi-honest") adversary ($\mathcal{T}$) that corrupts up to one of computing parties (P_0, P_1). We consider standard security model in line with the related works (Ibar-

Algorithm 5:

FunBic-CCA.eval $(a_{ij_d}, f_d, K_d, d) \rightarrow d_{ij_d}$.

Input: a_{ij_d} : input matrix

$(\in \{a_{ij_0}, a_{ij_1}\}, \mathbb{Z}_{2^n}^{l_r \cdot l_c})$.

f_d : similarity scores $(\in \{H_{ij_d}, H_{r_d}, H_{c_d}\})$.

K_d : preprocessing keys from setup phase.

d : computing parties $(\in \{0, 1\})$.

Output: d_{ij_d} : arithmetic shares of output matrix after deletion/ addition steps.

Evaluation Steps:

Preparation of input to FSS gate ($\hat{z}_{\gamma_d}$):

- ① $1_{r2del} : \alpha H_{ij_d} - H_{r_d} + r_{\gamma_d}$
- ② $1_{r2sdel} : H_{r_d}[d(i)] - H_{c_d}[d(j)] + r_{\gamma_d}$
- ③ $1_{r2add} : H_{ij_d} - H_{r_d} + r_{\gamma_d}$
- ④ $1_{stop} : H_{ij_d} - \delta + r_{\gamma_d}$
- ⑤ $1_{req} : I'_d - I_d + r_{\gamma_d}$

Reconstruction of masked input:

- ① $\hat{z}_{\gamma_d} \Rightarrow P_{1-d}; \hat{z}_{\gamma} \leftarrow \hat{z}_{\gamma_0} + \hat{z}_{\gamma_1}$

Comparison (IC) and equality check (DPF):

- ① $o_{geq_d} \leftarrow \text{FSS.Eval}^{IC}(d, K_d^{IC}, \hat{z}_{\gamma})$
- ② $o_{eq_d} \leftarrow \text{FSS.Eval}^{EQ}(d, K_d^{EQ}, \hat{z}_{\gamma})$

Algorithm 6:

FunBic-CCA.result $d_{ij_d} \rightarrow \text{Bic}_k(I, J)$.

Input: d_{ij_d} : output arithmetic shares.

Output: $\text{Bic}_k(I, J)$: k δ -biclusters.

Result Steps:

- ① $d_{ij} = d_{ij_0} + d_{ij_1}$
- ② **return** $\text{Bic}_k(I, J) = d_{ij}$

rondo et al., 2023; Boyle et al., 2021), having a static corruption model. Therefore, the adversary must choose a participant to corrupt prior to computations.

We simulate the corruption of a party P_d by resorting to the standard real world - ideal world paradigm (Canetti, 2001). The ideal world requires an additional trusted party that obtains all the inputs from the involved parties, which will then receive correct results by the correct computation of the ideal functionality. On the other hand, we execute the aforementioned protocols of the FunBic-CCA algorithm in the real world with $\mathcal{T}$. Accordingly, we begin with representing our ideal functionality of FunBic-CCA in FUNCTIONALITY 5 to prove that our security works in the $I_{\text{FunBic-CCA.setup-hybrid}}$ model. Thus, the model is based on the faithful execution of the FunBic-CCA.setup by our defined trusted party to provide designated parties with each piece of setup material.

FUNCTIONALITY 5 ($I_{\text{bic-identif}}(\mathbf{A}) \rightarrow \mathbf{D}$):

Upon receiving a share of input matrix $\mathbf{A}$, known parameters δ, α, k , preprocessing materials $\hat{a}_d, \hat{b}_d, \hat{e}_d, K^{IC}, K^{DPF}, r_\gamma$ from $P_d, d \in \{0, 1\}$, $I_{\text{bic-identif}}$ reconstructs $\hat{e}_d = r_{ij_d} - \hat{a}_d$, computes H_{ij}, H_r, H_c and $1_{r2del}, 1_{c2del}, 1_{r2sdel}, 1_{c2sdel}, 1_{r2add}, 1_{c2add}$, finally returns the shares of output matrix $\mathbf{D}$.

Security Proof. We assert that there is a Probabilistic Polynomial Time (PPT) key generation algorithm. Simulator ($\mathcal{S}$) realises the mentioned ideal functionality $I_{\text{bic-identif}}$ for each individual participant (given $\forall \mathbf{A} \in \mathbb{R}^{l_r \cdot l_c}, \forall \gamma \in \mathbb{Z}_{n^+}^*$) in which $\mathcal{S}$'s behaviour is computationally or statistically indistinguishable from a real world execution of our protocols 3, 5, and 6, having a semi-honest adversary $\mathcal{T}$.

Proof. We prove the security of our framework by considering $\mathcal{S}$ and possible scenarios during execution of the protocols:

- **FunBic-CCA.setup:** This offline phase can be seen as a black-box access in an ideal world that provides parties with preprocessing materials. We realise this setup phase as an ideal functionality $I_{\text{FunBic-CCA.setup}}$. Its simulation is grounded on the security of the underlying primitive used to instantiate it, generic 2PC for the FSS keys (Appendix A.2 of (Boyle et al., 2021)), and OT (Oblivious Transfer), HE (Homomorphic Encryption) for the SS preprocessing materials in (Demmler et al., 2015).
- **FunBic-CCA.MSR:** For the messages that a particular party is holding without sharing with the other one, nothing is needed to be considered by $\mathcal{S}$, because $\mathcal{T}$ does not have access to any messages. In the other cases in which P_{1-d} is the owner, $\mathcal{S}$ still executes the protocols honestly. Our local functions, including addition/ subtraction and division do not need to be simulated as of being done in a non-interactive way. The security proof of our secured multiplication is the same as for the Beaver-triple-based secure multiplication protocol.
- **FunBic-CCA.eval:** In the online phase, $\mathcal{S}$ honestly follows the protocol steps for finding the score using the data from the MSR phase. This is followed by the reconstruction of $\langle \hat{z}_\gamma \rangle$ as $\hat{z}_\gamma = \hat{z}_{\gamma_0} + \hat{z}_{\gamma_1}$ which is our input to FSS gates.

We argue computationally indistinguishability of the ideal world - real world executions for FSS gates based on (Boyle et al., 2021) (Definition 2).

In this simulation, the information in FSS keys K_0, K_1 as well as mask r_γ is preserved.

- **FunBic-CCA.result:** $\mathcal{S}$ simulates the output result with receiving $\langle o \rangle$ from $\mathcal{T}$ and o_{1-d} , associated with P_{1-d} , then can compute $o_d = 1 - o_{1-d}$. $\mathcal{S}$ provides $\mathcal{T}$ with o_d on behalf of P_{1-d} . After all, $\mathcal{S}$ has the output of P_d from $\mathcal{T}$.

Correctness Proof The execution of FunBic-CCA protocols satisfies:

$$\begin{aligned}
 & \text{if FunBic-CCA.setup}(l_r, l_c, n, \lambda, \gamma) \rightarrow K_0, K_1, \\
 & \text{and FunBic-CCA.MSR}(a_{ij_d}, K_d, d) \rightarrow f_0, f_1 \\
 & \text{then Pr[FunBic-CCA.eval}(a_{ij_0}, f_0, K_0, 0) \\
 & \quad + \text{FunBic-CCA.eval}(a_{ij_1}, f_1, K_1, 1) \\
 & = \{1_{r2del}(H_{r_d}, \alpha \cdot H_{ij_d}) > 0, 1_{c2del}(H_{c_d}, \alpha \cdot H_{ij_d}) > 0, \\
 & 1_{r2sdel}(H_{r_d}[d(i)], H_{c_d}[d(j)]) \geq 0, 1_{c2sdel}(H_{c_d}[d(j)], H_{r_d}[d(i)]) > 0 \\
 & 1_{r2add}(H_{r_d}, H_{ij_d}) \geq 0, 1_{c2add}(H_{c_d}, H_{ij_d}) \geq 0\} \rightarrow d_{ij_0}, d_{ij_1} = 1 \\
 & \quad (9)
 \end{aligned}$$

for threshold $\gamma (\in \mathbb{Z}_{n^+}^*)$, input matrix $\mathbf{A} (\in l_r \cdot l_c)$, and suitable choice of FSS as well as computing functions including deletion and addition.

Proof. Our proof on the correctness of IC gate is grounded on Theorem 3 of (Boyle et al., 2021). The two protocols $\text{FSS.Gen}^{IC}(\lambda, n, r)$, $\text{FSS.Eval}^{IC}(d, k_d^{IC}, \hat{z}_\gamma)$ establish the IC gate correctly, maintaining the condition of $f(z_\gamma, \gamma) = z_\gamma \geq \gamma$. Lastly, we argue what we mentioned earlier based on the Definition 2 of (Boyle et al., 2021) that

$$\begin{aligned}
 & \text{Pr}[\text{FSS.Eval}^{IC}(0, k_0^{IC}, \hat{z}_\gamma) + \\
 & \text{FSS.Eval}^{IC}(1, k_1^{IC}, \hat{z}_\gamma) = (z_\gamma \geq \gamma)] = 1 \quad (10)
 \end{aligned}$$

We analyse the correctness of the equality test according to Theorem 4.3 of (Boyle et al., 2019). The correctness can be seen since point function $f_{\alpha, \beta}(x)$ evaluates to $\beta = 1$ once $(x_1 - x_2) = \alpha = (r_1^{in} - r_2^{in})$ or having $(x_1 - r_1^{in}) = (x_2 - r_2^{in})$.

Afterwards, we prove the correctness of SS-based multiplication (i.e., r_{ij}^2) by resorting to the definition of scheme about random shares ($a_{ij_d} = a_{ij_0} + a_{ij_1}$), beaver multiplication triples ($\hat{a}_d, \hat{b}_d, e : \text{Dec}(\hat{e}_d)$) and having secret shares ($r_{ij_d} = r_{ij_0} + r_{ij_1}, \hat{e}_d = r_{ij_d} - \hat{a}_d$) after applying the local functions, including addition/ subtraction and secured division:

$$\begin{aligned}
& e^2 + (\hat{b}_0 + \hat{b}_1) + 2 \times e \times (\hat{a}_0 + \hat{a}_1) \\
&= e^2 + \hat{b} + 2 \times e \times \hat{a} = e^2 + \hat{a}^2 + 2 \times e \times \hat{a} = (e + \hat{a})^2 \\
&= [(r_{ij_1} - \hat{a}_1) + (r_{ij_0} - \hat{a}_0) + \hat{a}]^2 \\
&= [(r_{ij_1} - \hat{a}_1) + (r_{ij_0} - \hat{a}_0) + (\hat{a}_0 + \hat{a}_1)]^2 \\
&= [(r_{ij_1} + r_{ij_0}) - (\hat{a}_1 + \hat{a}_0) + (\hat{a}_0 + \hat{a}_1)]^2 \\
&= (r_{ij_1} + r_{ij_0})^2 = r_{ij}^2 \quad (11)
\end{aligned}$$

4 EXPERIMENT

In this Section, we represent our implementation of the FunBic-CCA framework, evaluation of resulting biclusters and finally summarise by discussing time performance and accuracy obtained.

Implementation and Environment. Our privacy-preserving implementation drives from Funshade (Ibarrondo et al., 2023) to construct the protocol steps (2, 3, 5, and 6) efficiently. Additionally, we use sycet (Ryffel et al., 2020) to implement only the equality checks. We also rely on biclustlib (Padilha and Campello, 2017) for the implementation of the original CCA, yeast cell cycle dataset and accuracy measures.

In the original algorithm (Cheng and Church, 2000), rows that form mirror images are added in bicluster throughout the node addition phase. However, they are not of interest when defining a bicluster in a general framework (Galvani et al., 2021). Accordingly, we change our focus in this work only on rows in the addition phase to be applicable to a broader set of data and increase the performance.

FunBic-CCA is implemented and executed on an Intel Core i7-1185 CPU, 3.00 GHz with 8 physical cores available and 31 GB system memory over at least 10 runs.

Datasets. We have utilised the yeast cell cycle and human expression data that were used for testing the original implementation of the algorithm (Cheng and Church, 2000). The yeast cell cycle expression data consists of 2884 genes and 17 conditions that were selected according to Tavazoie et al. (Tavazoie et al., 1999). The input matrix is of type integers in the range between 0 and 600. The human data consists of 4026 genes and 96 conditions that were reported according to Alizadeh et al. (Alizadeh et al., 2000). The data is a matrix of integers in a range between -750 and 650 with 47639 missing values (12.3 %) after

Table 3: Number of rounds and the communication size for one time execution of the FunBic-CCA protocol steps in the online phase. n is the bit size of the matrix elements, $l_r \cdot l_c$ is the matrix size.

Step	No. rounds	Communication size (bits)
MSR	1	$2n(l_r \cdot l_c)$
Eval	26	$4n[1 + 6(l_r + l_c)]$
Result	1	$2n(l_r \cdot l_c)$
Total	28	$4n[1 + (l_r \cdot l_c) + 6(l_r + l_c)]$

a scaling by a factor of 100. We replace these missing values of both data sets by a uniform distribution between 0 and 800.

Parameter Selection. According to the original study (Cheng and Church, 2000), the authors suggested finding of 100 biclusters with $\delta = 300$ and 1200 (maximum acceptable value for score) over the yeast cell cycle and human datasets, respectively. The multiple node deletion threshold (α) in their experiment was set to 1.2 for the datasets having a number of conditions below 100, like our used datasets to enhance the execution time.

In MPC, integers are naturally encoded into native data types, such as *uint32_t* or *uint64_t*, due to the implementation of protocols over finite rings or fields. We work with $n = 64$ -bit modular arithmetic with $\lambda = 128$, considering the scale of input matrices and maximum bit width during secure computations. We leverage the normalisation of the input matrices and integer division, while such bounds prevent from inherently impeding any secure computation protocol.

We assess maximum communication size and round of communication for each protocol step in the online phase, and list them in Table 3, given that our biclustering algorithm runs through $\log l_r + \log l_c$ iterations in the multiple node deletion, $l_r \cdot l_c$ rounds in the single node deletion with only one iterate of the node addition for each k bicluster (Cheng and Church, 2000). We consider here, one time execution of the online protocol steps, including FunBic-CCA.MSR with local functions (addition, subtraction, division) and one secured squaring (consists of one round of communications). Regarding FunBic-CCA.eval, our node deletion and addition steps rely on the reconstruction of the masked inputs to FSS gates with one round of communication.

Accuracy Measures. The quality of a bicluster needs to be assessed by evaluation functions aligned with biclustering algorithms. These external evaluation criteria measure how close FunBic-CCA is to the

ground truth. Thus, in this paper, we apply similarity measures such as Liu and Wang (Liu and Wang, 2007) and further extend it to the Prelic relevancy score (Prelic et al., 2006).

Considering $B \triangleq \{B_i\}_{i=1}^k$ and $\hat{B} \triangleq \{\hat{B}_i\}_{i=1}^q$ as the found and reference biclustering; Liu and Wang (lw) defined their similarity score between two biclusters as below (Horta and Campello, 2014):

$$lw(B, \hat{B}) \triangleq \frac{1}{k} \sum_{i=1}^k \max_{l \in \mathbb{N}_{1,q}} \left\{ \frac{|B_i^r \cap \hat{B}_l^r| + |B_i^c \cap \hat{B}_l^c|}{|B_i^r \cup \hat{B}_l^r| + |B_i^c \cup \hat{B}_l^c|} \right\} \quad (12)$$

Further we utilise the relevancy score derived from match score (MS) (Dale et al., 2018) between two sets of biclusters $\mathcal{S}_1$ and $\mathcal{S}_2$ for all biclusters $\mathcal{B}_1 \in \mathcal{S}_1$ with respect to biclusters $\mathcal{B}_2 \in \mathcal{S}_2$ as:

$$MS(\mathcal{S}_1, \mathcal{S}_2) = \frac{1}{|\mathcal{S}_1|} \sum_{\mathcal{B}_1 \in \mathcal{S}_1, \mathcal{B}_2 \in \mathcal{S}_2} \max \left\{ \frac{|\mathcal{B}_1 \cap \mathcal{B}_2|}{|\mathcal{B}_1 \cup \mathcal{B}_2|} \right\} \quad (13)$$

4.1 Results

Impact on Accuracy. Here, we study the accuracy of our solution based on the above-mentioned key measures, which range over the interval $[0, 1]$, with higher values indicating better solutions. Missing elements for both datasets are randomly replaced with values greater than or equal to 0. We choose $\alpha = 1.2$ and minimum number of the columns to be 100, according to the original study for both datasets (Cheng and Church, 2000). δ for experiments with yeast cell cycle and human gene expression data is selected based on the experiment by Tavazoie et al. (Tavazoie et al., 1999), whose reported clusters had scores in the range of between 261 and 996, with a median of 630.

Among the contributing parameters, n has a direct impact on the accuracy (see Figure 3). Lower numerical precision leads to a drop in accuracy; while its improvement also occurs as long as natural overflow is avoided and computation is not exceed $[0, 2^n - 1]$ (unsigned integers). Best accurate biclusters are derived from δ being close to the lower end of the range ($\delta = 300$) (Tavazoie et al., 1999) to detect more refined patterns. Because the size and the variance in the human data are doable and quadrupling compared to yeast, the high quality biclusters are achieved with $\delta = 1200$. In accordance to the findings of (Liu and Wang, 2007; Prelic et al., 2006), when the number of the biclusters (k) is small, the match scores decrease; because CCA itself is not powerful enough when dealing with small set of biclusters. To conclude, we record the 100% correctness, when inputting both datasets, for 100 biclusters, and $n = 32$ -bit.

Table 4: Computation and communication overhead of FunBic-CCA (online phase) for each actor over yeast cell cycle with $\delta = 300$.

n	Time (s)		Bandwidth (KB)	
	DT	P_d	$DT \leftrightarrow P_d$	$P_{1-d} \leftrightarrow P_d$
8	14.2383	123.6316	1221.86	400.9778
16	14.2110	123.4150	1370.52	400.9453
32	14.2907	125.4250	1371.9	400.9676
64	14.1132	126.1317	1376.26	403.0315

Performance analysis In this Section, we analyse the performance of FunBic-CCA. According to Figure 4, we notice that by enlarging the bit width of the input matrix, both latency and bandwidth in the online phase are increased. We set the original values of δ for the yeast ($\delta = 300$) and human data ($\delta = 1200$).

Parties evaluate “FunBic-CCA.MSR” jointly $\cong 42$ seconds (s), and $\cong 110$ (s) on yeast and human data. We record on average 96.8225 (s) having yeast cell cycle, while 345.64 (s) on human data for another step in the online phase, “FunBic-CCA.eval”. We extend the analyses to test how other parameters, including k and δ can affect the overall performance in the online phase. In this regard, k linearly increases the latency, while lowering δ links to the higher number of computations inside the algorithm, thus increases the latency (see Figure 5).

We utilise a secure channel with capacity of 100 Mbps for inter and intra-protocol communication, where a LAN setting has 10ms and WAN 70ms of transmission latency. The results in Figure 6 include total online communication latency, given the costs in Table 3 for a LAN or WAN setting and the computation time in Figure 4.

In addition, we time the total latency by each actor, including the data owner plus trusted dealer (DT), and the parties, in Table 4 and 5. Timings for parties consist of the three protocol steps “FunBic-CCA.MSR”, “FunBic-CCA.eval” and “FunBic-CCA.result” to identify the scores, perform deletion/ addition on the nodes, then send back the secret shared of the outputs. We determine the total latency of the DT, which is referred to “FunBic-CCA.setup” and reconstruction of the biclusters. Almost 89.93% and 91.81% of the total execution time allocate to the parties performing the protocol steps in the online phase, remaining the rest for the DT on 64-bits yeast cell cycle and human expression data respectively.

Besides, the communication size between the DT and parties is calculated to less than 1380 kilobytes (kb) for both datasets. According to Figure 2, the communicated elements are the secret shares of the

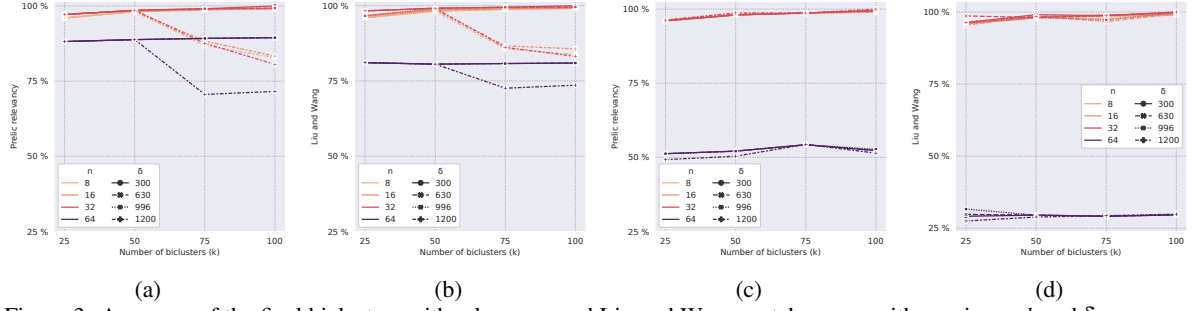


Figure 3: Accuracy of the final biclusters with relevance, and Liu and Wang match scores with varying n , k and δ over yeast cell cycle (a, b) and human expression data (c, d).

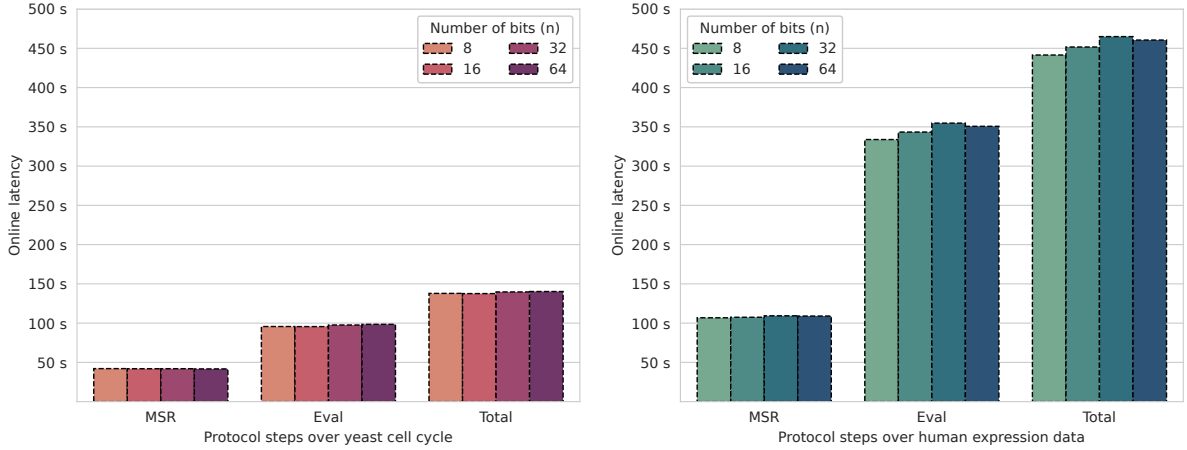


Figure 4: Computation and communication overhead of FunBic-CCA for each protocol step in the online phase over yeast cell cycle ($\delta = 300$) and human expression data ($\delta = 1200$).

Table 5: Computation and communication overhead of FunBic-CCA (online phase) for each actor over human data with $\delta = 1200$.

n	Time (s)		Bandwidth (KB)	
	DT	P_d	$DT \leftrightarrow P_d$	$P_{1-d} \leftrightarrow P_d$
8	36.9292	404.5446	1224.74	2247.0354
16	37.1300	414.5372	1373.26	2247.9475
32	37.7045	427.2254	1373.17	2247.7641
64	37.6850	422.8491	1380.47	2251.3945

input matrix (a_{ij_0}, a_{ij_1}) , known parameters for our algorithm (δ, α, k) and the preprocessing materials (i.e., K_0, K_1 or the correlated randomness and FSS keys). On the other hand, the parties send back the secret shares of the output matrix to the data owner for its reconstruction. We also record the communication size between the two computing parties, exchanging the masked secret shares in performing multiplication (refer to Section 3.1) and the comparative functions (see Table 2). On average, 2248.5353 (kb) is communicated between two computing parties for human data, which is approximately $\times 5$ higher than the size

of communication in yeast data due to the difference in their range and size.

5 RELATED WORK

Personal data misuse and theft, especially when dealing with patients' genomic data, are ever-present concerns due to carrying highly private information. Accordingly, privacy-compliant patient data analyses are among typical applications of privacy-preserving computation techniques.

A numerous of these privacy enhancing techniques are utilised for supervised machine learning methods leaving out other approaches, including clustering. Only in the field of secure multi-party computation has the majority of research been carried out covering neural networks that are often followed by ReLU activations (Boemer et al., 2020; Riazi et al., 2018), grounded on a mixture of Garbled Circuits, secret sharing and their conversions. In addition, mixed mode protocols (Demmler et al., 2015; Patra et al., 2021) for secure two-party computation relies on a

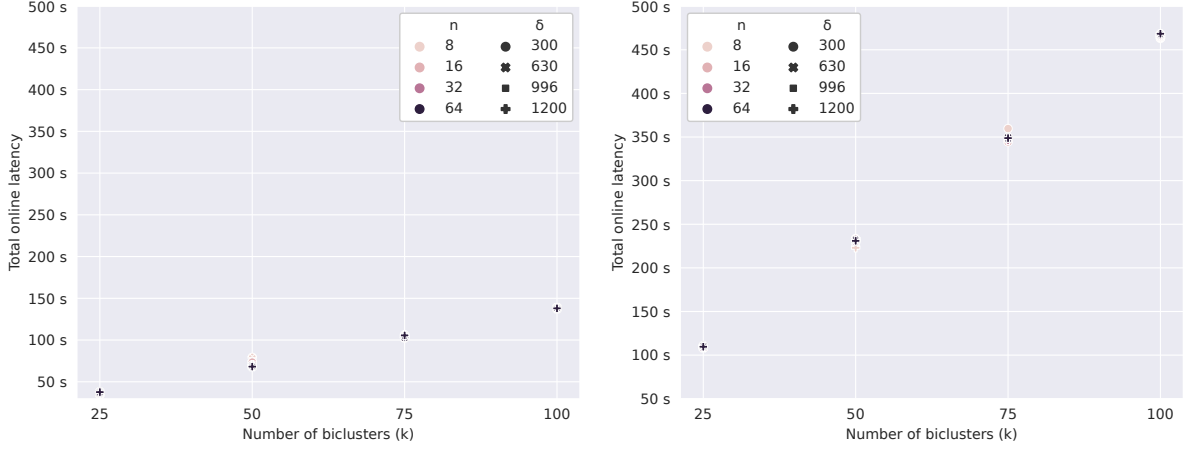


Figure 5: Total online latency with varying n , k and δ over yeast cell cycle (left) and human expression data (right).

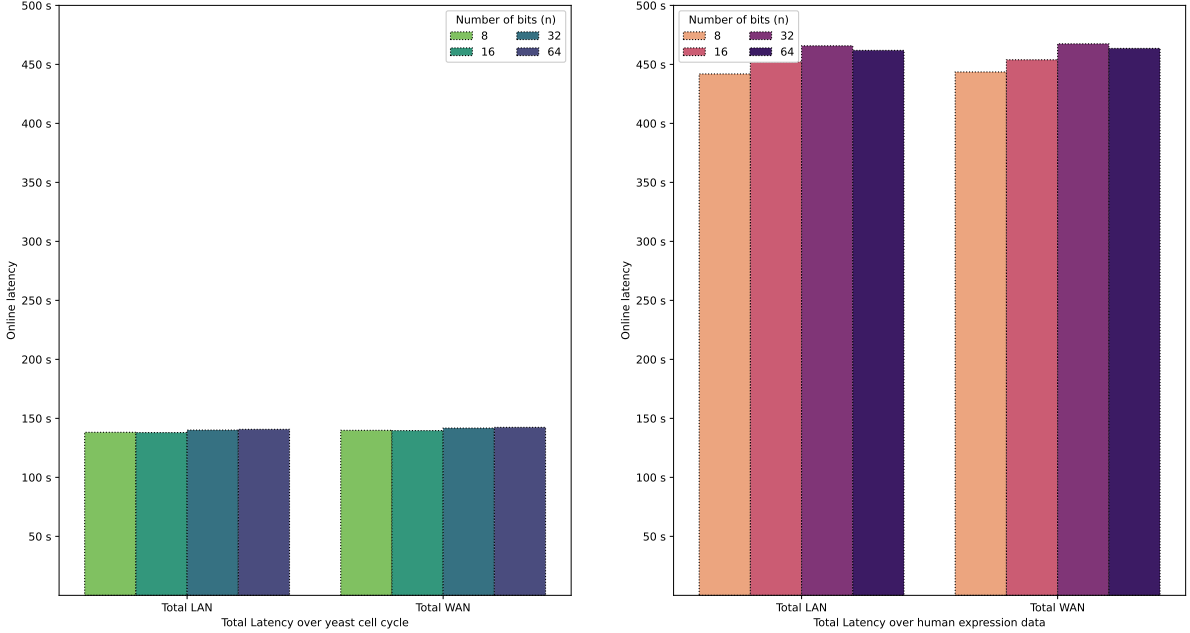


Figure 6: Total latency (communication + computation) for the online execution of FunBic-CCA over the yeast cell cycle ($\delta = 300$, on the left) and human expression data ($\delta = 1200$, on the right) in the LAN and WAN settings.

considerable communication cost to evaluate comparisons. In line with works on FSS to name but few (Boyle et al., 2015; Boyle et al., 2019; Boyle et al., 2021; Ryffel et al., 2020; Ibarrondo et al., 2023), these solutions incur promising results leading to our framework. Table 6 represents recent FSS-based secure frameworks. Regardless of the classification methods in machine learning, another approach based on clustering is gaining more attention to be performed in a privacy-preserving manner (Mohassel et al., 2020; Meng et al., 2019). We refer avid reader to (Hegde et al., 2021) for more details. Regarding unsupervised machine learning algorithm

based on biclustering, there exist methods in the literature that use searches reliant on traditional one-way clustering and combine additional techniques to analyse the second dimension (Fraiman and Li, 2020; Ardanewari et al., 2017). For instance, clustering based on the singular value decomposition whose privacy is protected mainly by data distortion (Maheswari and Duraiswamy, 2008; Lakshmi and Rani, 2013). Moreover, co-clustering algorithms with matrix factorisation are widely used in text clustering and gene expression analysis demonstrated their superiority to traditional, one-side clustering (Lin et al., 2019). Their privacy has been discussed in several

recent works, particularly when making use of differential privacy (Li et al., 2020; Guo et al., 2023).

As mentioned earlier, a few of the recent works determined the importance of preserving the privacy of biclustering methods that are more adaptable approaches to gene expression data whose information loss could have detrimental impacts on the data owner. Ahmad et al. (Ahmad and Khokhar, 2007a; Ahmad and Khokhar, 2007b) proposed a privacy-preserving collaborative filtering framework using biclustering. They employed a bigraph crossing minimisation based biclustering algorithm and ensured privacy of the users by secure multi-party computations that were carried out using additively homomorphic cryptosystems. Unfortunately, the papers neither targeted the expression data by biclustering methods such as Cheng and Church (Cheng and Church, 2000) as one of the most cited and used algorithms nor building upon recent advances in FSS. Our proposed solution can be adaptable to a range of MSR-based algorithms.

In the field of Homomorphic Encryption, privacy-preserving machine learning use case has led to a variety of approaches (Kim and Lauter, 2015; Blatt et al., 2020) to encrypt model in an untrusted cloud storage over encrypted data. Recent works in the family of privacy-preserving biclustering expression data, from which we draw inspiration, are SecBic-CCA, and SeCCA (VahidianSadegh et al., 2022) that our solution improves in different aspects. Both of these frameworks are implemented by using Homomorphic Encryption schemes including CKKS (Cheon et al., 2017) and BFV (Fan and Vercauteren, 2012). These approaches do not include comparison to a threshold, and rely on decryption of intermediate results with a considerable latency time and reduced accuracy.

6 CONCLUSIONS

In this paper, we introduced our framework upon an additive secret sharing scheme and function secret sharing for a full correctness for comparison with a fixed threshold in an online phase between two computing parties. Thanks to the proposed solution, FunBic-CCA is the first protocol that applies function secret sharing over Cheng and Church Algorithm and discloses similarity score, all while relying on lightweight cryptographic primitives. We implement our solution on top of the open-source libraries and showcase its 100% correctness with 32-bit precision and latency within $\cong 500s$ against 4026 rows and 96 columns. Future research is envisioned for extending FunBic-CCA to guarantee security with abort against

malicious adversaries, using MACs (i.e., a primitive that is used to guarantee data integrity and authenticate the parties' shares).

REFERENCES

- Ahmad, W. and Khokhar, A. (2007a). An architecture for privacy preserving collaborative filtering on web portals. In *Third International Symposium on Information Assurance and Security*, pages 273–278. IEEE.
- Ahmad, W. and Khokhar, A. (2007b). Privacy preserving collaborative filtering using biclustering in ubiquitous computing environments. In *Proceedings of Fifth International Workshop on Databases, Information Systems and Peer-to-Peer Computing, Vienna, Austria*. Citeseer.
- Ali, A., Ajil, A., Meenakshi Sundaram, A., and Joseph, N. (2023). Detection of gene ontology clusters using biclustering algorithms. *SN Computer Science*, 4(3):217.
- Alizadeh, A. A., Eisen, M. B., Davis, R. E., Ma, C., Lossos, I. S., Rosenwald, A., Boldrick, J. C., Sabet, H., Tran, T., Yu, X., et al. (2000). Distinct types of diffuse large b-cell lymphoma identified by gene expression profiling. *Nature*, 403(6769):503–511.
- Ardaneswari, G., Bustamam, A., and Siswantining, T. (2017). Implementation of parallel k-means algorithm for two-phase method biclustering in carcinoma tumor gene expression data. In *AIP Conference Proceedings*, volume 1825. AIP Publishing.
- Beaver, D. (1992). Efficient multiparty protocols using circuit randomization. In *Advances in Cryptology—CRYPTO’91: Proceedings 11*, pages 420–432. Springer.
- Blatt, M., Gusev, A., Polyakov, Y., and Goldwasser, S. (2020). Secure large-scale genome-wide association studies using homomorphic encryption. *Proceedings of the National Academy of Sciences*, 117(21):11608–11613.
- Boemer, F., Cammarota, R., Demmler, D., Schneider, T., and Yalame, H. (2020). Mp2ml: A mixed-protocol machine learning framework for private inference. In *Proceedings of the 15th international conference on availability, reliability and security*, pages 1–10.
- Boyle, E., Chandran, N., Gilboa, N., Gupta, D., Ishai, Y., Kumar, N., and Rathee, M. (2021). Function secret sharing for mixed-mode and fixed-point secure computation. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 871–900. Springer.
- Boyle, E., Gilboa, N., and Ishai, Y. (2015). Function secret sharing. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 337–367. Springer.
- Boyle, E., Gilboa, N., and Ishai, Y. (2019). Secure computation with preprocessing via function secret sharing. In *Theory of Cryptography: 17th International Conference, TCC 2019, Nuremberg, Germany, December*

Table 6: Overview of FSS-based secure frameworks. $\odot$ indicates semi-honest security model, and $\bullet$: malicious security model. Primitives include Secret Sharing (SS), Function Secret Sharing (FSS), Replicated SS (RSS), optimized SS (o-SS), Distributed Comparison Function (DCF), Interval Containment (IC). Online computation blocks are considered for both linear and non-linear functions: Matrix Multiplication (MatMult), Convolution (Conv), Comparison (Comp), Feature Selection (FeatSelect), Path Evaluation (PathEval).

Work	Online Computation Blocks	Type	Parties	Security	Remarks
Funshade (Ibarrondo et al., 2023)	Scalar product, Comp. ($\geq \theta$)	o-SS, FSS (IC gate)	2PC	$\odot$	Optimised online phase ¹
Nomadic (Cheng et al., 2024b)	Cosine similarity, Comp. ($\geq \theta$)	o-SS, CondEval ²	2PC	$\bullet$	Optimised round of comm.
AriaNN (Ryffel et al., 2020)	MatMult., Conv., Argmax, MaxPool	SS, FSS (DCF gate)	2PC	$\odot$	Reduced FSS key size
FssNN (Yang et al., 2023)	MatMult., ReLU, DReLU, MaxPool	SS, FSS (DCF gate)	2PC	$\odot$	Reduced FSS key size
PDTE (Cheng et al., 2024a)	FeatSelect, Comp., PathEval	RSS, FSS (IC, DPF gates)	3PC	$\odot$	Constant rounds of comm.
LLAMA (Gupta et al., 2022)	MatMult., Conv., Sigmoid, Tanh	FSS (G_{smult} ³ , IC gates)	2PC	$\odot$	Mixed bitwidth operations
Orca (Jawalkar et al., 2024)	MatMult., Conv., ReLU, MaxPool	FSS (G_{smult} , DCF gates)	2PC	$\odot$	GPU acceleration
Waldo (Dauterman et al., 2022)	Additive and arbitrary aggregate	RSS, FSS (DCF, DPF gates)	3PC	$\bullet$	Multi-predicate filtering
FunBic-CCA (OURS)	MatMult., Comp. ($\geq \theta$), Argmax	SS, FSS (IC, DPF gates)	2PC	$\odot$	1st end-to-end secure CCA

¹ By merging local addition and one round of communication in o-SS with the same operations at the input of FSS.

² Allows computing the composition of an input bit and a binary function on an input.

³ Is an FSS gate used for signed multiplication.

1–5, 2019, *Proceedings, Part I* 17, pages 341–371. Springer.

- Canetti, R. (2001). Universally composable security: A new paradigm for cryptographic protocols. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pages 136–145. IEEE.
- Catalano, D., Cramer, R., Di Crescenzo, G., Darmgård, I., Pointcheval, D., Takagi, T., Cramer, R., and Damgård, I. (2005). Multiparty computation, an introduction. *Contemporary cryptography*, pages 41–87.
- Cheng, N., Gupta, N., Mitrokotsa, A., Morita, H., and Tozawa, K. (2024a). Constant-round private decision tree evaluation for secret shared data. *Proceedings on Privacy Enhancing Technologies*.
- Cheng, N., Önen, M., Mitrokotsa, A., Chouchane, O., Todisco, M., and Ibarrondo, A. (2024b). Nomadic: Normalising maliciously-secure distance with cosine similarity for two-party biometric authentication. In *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security*, pages 257–273.
- Cheng, Y. and Church, G. M. (2000). Biclustering of expression data. In *Ismb*, volume 8, pages 93–103.
- Cheon, J. H., Kim, A., Kim, M., and Song, Y. (2017). Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology—ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I* 23, pages 409–437. Springer.
- Dale, J., Nishimoto, A., and Obafemi-Ajayi, T. (2018). Performance evaluation and enhancement of biclustering

algorithms. In Marsico, M. D., di Baja, G. S., and Fred, A. L. N., editors, *Proceedings of the 7th International Conference on Pattern Recognition Applications and Methods, ICPRAM 2018, Funchal, Madeira - Portugal, January 16-18, 2018*, pages 202–213. SciTePress.

- Dauterman, E., Rathee, M., Popa, R. A., and Stoica, I. (2022). Waldo: A private time-series database from function secret sharing. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 2450–2468. IEEE.
- Demmler, D., Schneider, T., and Zohner, M. (2015). ABya framework for efficient mixed-protocol secure two-party computation. In *NDSS*.
- Escudero, D. (2022). An introduction to secret-sharing-based secure multiparty computation. *Cryptology ePrint Archive*.
- Fan, J. and Vercouteren, F. (2012). Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*.
- Fraiman, N. and Li, Z. (2020). Biclustering with alternating k-means. *arXiv preprint arXiv:2009.04550*.
- Galvani, M., Torti, A., Menafoglio, A., and Vantini, S. (2021). Funcc: A new bi-clustering algorithm for functional data with misalignment. *Computational Statistics & Data Analysis*, 160:107219.
- Gilboa, N. and Ishai, Y. (2014). Distributed point functions and their applications. In *Advances in Cryptology—EUROCRYPT 2014: 33rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Copenhagen, Denmark, May 11-15, 2014. Proceedings* 33, pages 640–658. Springer.
- Goldreich, O., Micali, S., and Wigderson, A. (1987). How to play any mental game or A completeness theorem

- for protocols with honest majority. In Aho, A. V., editor, *Proceedings of the 19th Annual ACM Symposium on Theory of Computing*, 1987, New York, New York, USA, pages 218–229. ACM.
- Guo, X., Li, X., Chang, X., and Ma, S. (2023). Privacy-preserving community detection for locally distributed multiple networks. *arXiv preprint arXiv:2306.15709*.
- Gupta, K., Kumaraswamy, D., Chandran, N., and Gupta, D. (2022). Llama: A low latency math library for secure inference. *Cryptology ePrint Archive*.
- Hartigan, J. A. (1972). Direct clustering of a data matrix. *Journal of the american statistical association*, 67(337):123–129.
- Hegde, A., Möllering, H., Schneider, T., and Yalame, H. (2021). Sok: Privacy-preserving clustering. In *NeurIPS 2021 Workshop Privacy in Machine Learning*.
- Horta, D. and Campello, R. J. (2014). Similarity measures for comparing biclusterings. *IEEE/ACM transactions on computational biology and bioinformatics*, 11(5):942–954.
- Ibarrondo, A., Chabanne, H., and Önen, M. (2023). Funshade: Function secret sharing for two-party secure thresholded distance evaluation. *Proceedings on Privacy Enhancing Technologies*.
- Jawalkar, N., Gupta, K., Basu, A., Chandran, N., Gupta, D., and Sharma, R. (2024). Orca: Fss-based secure training and inference with gpus. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 597–616. IEEE.
- Kim, M. and Lauter, K. (2015). Private genome analysis through homomorphic encryption. In *BMC medical informatics and decision making*, volume 15, pages 1–12. Springer.
- Knott, B., Venkataraman, S., Hannun, A., Sengupta, S., Ibrahim, M., and van der Maaten, L. (2021). Crypten: Secure multi-party computation meets machine learning. *Advances in Neural Information Processing Systems*, 34:4961–4973.
- Lakshmi, M. N. and Rani, K. S. (2013). Svd based data transformation methods for privacy preserving clustering. *International Journal of Computer Applications*, 78(3).
- Li, J., Wei, J., Ye, M., Liu, W., and Hu, X. (2020). Privacy-preserving constrained spectral clustering algorithm for large-scale data sets. *IET Information Security*, 14(3):321–331.
- Lin, R., Wang, S., and Guo, W. (2019). An overview of co-clustering via matrix factorization. *IEEE Access*, 7:33481–33493.
- Liu, X. and Wang, L. (2007). Computing the maximum similarity bi-clusters of gene expression data. *Bioinformatics*, 23(1):50–56.
- Maheswari, N. and Duraiswamy, K. (2008). Clust-svd: Privacy preserving clustering in singular value decomposition. *World Journal of Modelling and Simulation*, 4(4):250–256.
- Meng, X., Papadopoulos, D., Oprea, A., and Triandopoulos, N. (2019). Private two-party cluster analysis made formal & scalable. *arXiv preprint arXiv:1904.04475 v2*.
- Mohassel, P., Rosulek, M., and Trieu, N. (2020). Practical privacy-preserving k-means clustering. *Proceedings on privacy enhancing technologies*.
- Nawaz, M., Gulati, A., Liu, K., Agrawal, V., Ananth, P., and Gupta, T. (2020). Accelerating 2pc-based ml with limited trusted hardware. *arXiv preprint arXiv:2009.05566*.
- Padilha, V. A. and Campello, R. J. (2017). A systematic comparative evaluation of biclustering techniques. *BMC bioinformatics*, 18:1–25.
- Patra, A., Schneider, T., Suresh, A., and Yalame, H. (2021). ABY2. 0: Improved Mixed-Protocol secure Two-Party computation. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2165–2182.
- Prelić, A., Bleuler, S., Zimmermann, P., Wille, A., Bühlmann, P., Gruissem, W., Hennig, L., Thiele, L., and Zitzler, E. (2006). A systematic comparison and evaluation of biclustering methods for gene expression data. *Bioinformatics*, 22(9):1122–1129.
- Riazi, M. S., Weinert, C., Tkachenko, O., Songhori, E. M., Schneider, T., and Koushanfar, F. (2018). Chameleon: A hybrid secure computation framework for machine learning applications. In *Proceedings of the 2018 on Asia conference on computer and communications security*, pages 707–721.
- Ryffel, T., Tholoniati, P., Pointcheval, D., and Bach, F. (2020). Ariann: Low-interaction privacy-preserving deep learning via function secret sharing. *arXiv preprint arXiv:2006.04593*.
- Tavazoie, S., Hughes, J. D., Campbell, M. J., Cho, R. J., and Church, G. M. (1999). Systematic determination of genetic network architecture. *Nature genetics*, 22(3):281–285.
- VahidianSadeh, S., Ibarrondo, A., and Wiese, L. (2025). Funbic-cca: Function secret sharing for biclusterings applied to cheng and church algorithm. In *Proceedings of the 22nd International Conference on Security and Cryptography - Volume 1: SECRYPT*, pages 37–48. INSTICC, SciTePress.
- VahidianSadeh, S., Wiese, L., and Brenner, M. (2022). Secca: Towards privacy-preserving biclustering algorithm with homomorphic encryptions. In *IFIP International Summer School on Privacy and Identity Management*, pages 198–213. Springer.
- Wagh, S. (2022). Pika: Secure computation using function secret sharing over rings. *Proceedings on Privacy Enhancing Technologies*.
- Yang, P., Jiang, Z. L., Gao, S., Wang, H., Zhou, J., Jin, Y., Yiu, S.-M., and Fang, J. (2023). Fssnn: communication-efficient secure neural network training via function secret sharing. *Cryptology ePrint Archive*.
- Yao, A. C. (1982). Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*, pages 160–164. IEEE.